

PROGETTAZIONE DI SISTEMI DIGITALI

Michele De Fusco, Luigi Lo Bianco, Stefano Scarcelli, Costanza Ferrari

Relazione del progetto

Indice

1	Introduzione	2
2	Il moltiplicatore	4
2.1	Implementazione	4
2.1.1	Alcune note sui prodotti	6
2.2	Il testing	7

Capitolo 1

Introduzione

La traccia richiedeva l'implementazione di un circuito per il filtraggio di un'immagine greyscale di almeno 32x32 pixel rappresentati su 8 bit unsigned.

Il filtro deve essere composto da 3x3 coefficienti (scelti in modo arbitrario dal gruppo di lavoro) rappresentati su 4 bit in complemento a 2.

La finestra di convoluzione ha la stessa dimensione del filtro, quindi 3x3 e scorre lungo tutta l'immagine per filtrare un pixel alla volta; il procedimento di invio dei dati e riempimento del buffer line è gestito dalla state machine attraverso alcuni segnali che verranno descritti successivamente.

L'operazione di filtraggio deve essere condotta utilizzando un anchor point centrale quindi il pixel che subisce l'operazione di filtraggio è quello che si trova al centro della finestra.

L'immagine viene gestita attraverso il Buffer Line che restituisce i pixel di 3 righe allineati, pronti per essere inviati al moltiplicatore.

Viene rivolta particolare attenzione per i pixel che si trovano lungo i bordi e che nella finestra di convoluzione subiscono zero padding, che viene garantito dall'inizializzazione a 0 del buffer e dei vettori del componente.

Il filtraggio vero e proprio avviene attraverso l'operazione di convoluzione: i pixel dell'immagine vengono moltiplicati con il coefficiente corrispondente del filtro e infine tutti i prodotti parziali vengono sommati insieme. Il risultato sarà il pixel centrale filtrato con il Filtro Gaussiano o Gaussian Blur che ha come coefficienti:

```
2  1  2
2  4  2
1  2  1
```

Per il testing abbiamo optato per diverse immagini, tra cui alcune figure semplici che fanno risaltare molto l'influenza del filtro.

Per processare le immagini sono stati scritti diversi script MATLAB tra cui

- uno per creare semplici immagini da impiegare nel circuito;
- uno per ridimensionare immagini (prese da internet) nel formato 32x32 pixel;
- uno per estrarre le componenti numeriche di ogni immagine che saranno poi pronte per essere convertite in bit e inserite come input nel circuito;
- uno per rielaborare le componenti numeriche e ricostruire l'immagine filtrata.

Capitolo 2

Il moltiplicatore

2.1 Implementazione

La differenza principale con il moltiplicatore di Booth è che non si va a suddividere in gruppi di bit il moltiplicatore, ma si vanno a codificare tutti i possibili casi che possono accadere.

La nostra implementazione va a sfruttare i $2^4 = 16$ casi possibili per ogni coefficiente del filtro $F_{x,y}$.

La buffer line scorre sull'immagine e invia i segnali al moltiplicatore: i coefficienti del filtro sono sempre gli stessi, che poi vengono codificati nel medesimo modo, e i pixel dell'immagine vengono inviati 9 alla volta secondo una finestra 3x3.

Di seguito la entity:

```
entity booth_multiplier is
  generic(
    componente_immagine : POSITIVE := 8;
    coefficiente_filtro : POSITIVE := 4;
    somma : POSITIVE := 12
  );
  port (
    P_1_1, P_1_2, P_1_3 : in std_logic_vector(componente_immagine-1 downto 0);
    P_2_1, P_2_2, P_2_3 : in std_logic_vector(componente_immagine-1 downto 0);
    P_3_1, P_3_2, P_3_3 : in std_logic_vector(componente_immagine-1 downto 0);

    F_1_1, F_1_2, F_1_3 : in std_logic_vector(coefficiente_filtro-1 downto 0);
    F_2_1, F_2_2, F_2_3 : in std_logic_vector(coefficiente_filtro-1 downto 0);
    F_3_1, F_3_2, F_3_3 : in std_logic_vector(coefficiente_filtro-1 downto 0);

    M_1_1, M_1_2, M_1_3 : out std_logic_vector(somma-1 downto 0);
    M_2_1, M_2_2, M_2_3 : out std_logic_vector(somma-1 downto 0);
    M_3_1, M_3_2, M_3_3 : out std_logic_vector(somma-1 downto 0)
  );
end entity booth_multiplier;
```

Le entrate si suddividono in questo modo:

- i segnali $P_{x,y}$ sono i pixel dell'immagine che hanno dimensione 8 bit
- i segnali $F_{x,y}$ sono i coefficienti del filtro con dimensione 4 bit

Se il moltiplicando e il moltiplicatore sono composti rispettivamente da m ed n bit il risultato del prodotto andrà espresso in $m + n$ bit.

Le uscite sono invece i segnali $M_{x,y}$ con dimensione $m + n = 8 + 4 = 12$ bit.

Il moltiplicatore segue questo funzionamento:

- L'operazione principale che abbiamo effettuato è la codifica dei 16 possibili valori che può assumere ogni coefficiente del filtro;
- Si codificano i 2 prodotti parziali $P_P_A_{x,y}$ e $P_P_B_{x,y}$ per ogni pixel e relativo coefficiente. Dato l'elevato numero dei casi possibili diventa complesso offrire una codifica diretta quindi si utilizzano A e B che vengono ciascuno moltiplicati per $P_{x,y}$ separatamente ottenendo i segnali $P_P_A_{x,y}$ e $P_P_B_{x,y}$ secondo la seguente tabella:

Case filtro	Numero da codificare	A	B	$P_P_A_{x,y}$	$P_P_B_{x,y}$
0000	0	0	0	000000000000	000000000000
0001	1	1	0	0000 $P_{x,y}$	000000000000
0010	2	-2	4	$N_P_{x,y}(8)$ $N_P_{x,y}(8)$ $N_P_{x,y}$ 0	00 $P_{x,y}$ 00
0011	3	-1	4	$N_P_{x,y}(8)$ $N_P_{x,y}(8)$ $N_P_{x,y}(8)$ $N_P_{x,y}$	00 $P_{x,y}$ 00
0100	4	0	4	000000000000	00 $P_{x,y}$ 00
0101	5	1	4	0000 $P_{x,y}$	00 $P_{x,y}$ 00
0110	6	-2	8	$N_P_{x,y}(8)$ $N_P_{x,y}(8)$ $N_P_{x,y}$ 0	0 $P_{x,y}$ 000
0111	7	-1	8	$N_P_{x,y}(8)$ $N_P_{x,y}(8)$ $N_P_{x,y}(8)$ $N_P_{x,y}$	0 $P_{x,y}$ 000
1000	-8	0	-8	000000000000	$N_P_{x,y}$ 000
1001	-7	1	-8	0000 $P_{x,y}$	$N_P_{x,y}$ 000
1010	-6	-2	-4	$N_P_{x,y}(8)$ $N_P_{x,y}(8)$ $N_P_{x,y}$ 0	$N_P_{x,y}(8)$ $N_P_{x,y}$ 00
1011	-5	-1	-4	$N_P_{x,y}(8)$ $N_P_{x,y}(8)$ $N_P_{x,y}(8)$ $N_P_{x,y}$	$N_P_{x,y}(8)$ $N_P_{x,y}$ 00
1100	-4	0	-4	000000000000	$N_P_{x,y}(8)$ $N_P_{x,y}$ 00
1101	-3	1	-4	0000 $P_{x,y}$	$N_P_{x,y}(8)$ $N_P_{x,y}$ 00
1110	-2	-2	0	$N_P_{x,y}(8)$ $N_P_{x,y}(8)$ $N_P_{x,y}$ 0	000000000000
1111	-1	-1	0	$N_P_{x,y}(8)$ $N_P_{x,y}(8)$ $N_P_{x,y}(8)$ $N_P_{x,y}$	000000000000

Da notare che $N_P_x_y$ ha dimensione 9 bit ed esprime la negazione del pixel P_x_y , quindi corrisponde alla sua inversione dei singoli bit e alla somma di '1' attraverso una batteria di Ripple Carry Adder, uno per ogni pixel e si utilizza quando i prodotti parziali $P_P_A_x_y$ oppure $P_P_B_x_y$ sono negativi.

Da ricordare che $P_P_A_x_y$ e $P_P_B_x_y$ devono essere di 12 bit quindi vanno gestiti bene i prodotti da effettuare attraverso gli shift sinistri e l'estensione che deve portare ogni segnale alla dimensione giusta.

Questa operazione all'interno del codice risulta in 9 case statement in cui si codificano le componenti $P_P_A_x_y$ e $P_P_B_x_y$ per tutti i coefficienti del filtro.

- L'ultimo passaggio è la somma fra le componenti $P_P_A_x_y$ e $P_P_B_x_y$ calcolata per tutti gli ingressi attraverso dei semplici circuiti sommatore Ripple Carry.

2.1.1 Alcune note sui prodotti

I prodotti vengono effettuati in modo intelligente, provando a sfruttare operazioni immediate come shift sinistri e negazioni, quindi quelli per 0, +1, -1, +2, -2, +4, -4 e così via, nello stesso modo in cui si calcolano nel moltiplicatore di Booth.

Questi prodotti sono semplici da ottenere:

- Prodotto per 0 => Il prodotto parziale avrà tutti i bit a 0;
- Prodotto per 1 => Il prodotto parziale sarà uguale al moltiplicando;
- Prodotto per -1 => Il prodotto parziale sarà uguale al moltiplicando che subisce l'inversione dei bit e a cui viene sommato 1;
- Prodotto per 2 => Il prodotto parziale sarà uguale al moltiplicando che subisce uno shift sinistro;
- Prodotto per -2 => Il prodotto parziale sarà uguale al moltiplicando che subisce l'inversione dei bit, la somma di 1 e uno shift sinistro;
- Prodotto per 4 => Il prodotto parziale sarà uguale al moltiplicando che subisce due shift sinistri;

- Prodotto per -4 => Il prodotto parziale sarà uguale al moltiplicando che subisce l'inversione dei bit, la somma di 1 e due shift sinistri; e così via.

La strategia è quella di utilizzare A e B per ottenere i prodotti non immediati (ad esempio 3, 5, 6, 7 e così via) combinando i prodotti parziali $P_P_A_x_y$ e $P_P_B_x_y$ attraverso la somma finale.

Ad esempio nel caso in cui il coefficiente abbia valore 5 (quindi 0101) si ricorre a codificare il prodotto per A=1 e per B=4, si determinano i parziali da sommare insieme e si avrà ottenuto il pixel filtrato.

2.2 Il testing

Si allegano ora i risultati del testbench effettuato in cui figurano i risultati correttamente calcolati:

> ♥ P_1_1[7:0]	255	0	255	0	255														
> ♥ P_1_2[7:0]	255	0	255	0	255														
> ♥ P_1_3[7:0]	255	0	255	0	255														
> ♥ P_2_1[7:0]	255	0	255	0	255														
> ♥ P_2_2[7:0]	255	0	255	0	255														
> ♥ P_2_3[7:0]	255	0	255	0	255														
> ♥ P_3_1[7:0]	255	0	255	0	255														
> ♥ P_3_2[7:0]	255	0	255	0	255														
> ♥ P_3_3[7:0]	255	0	255	0	255														
> ♥ F_1_1[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2									
> ♥ F_1_2[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2									
> ♥ F_1_3[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2									
> ♥ F_2_1[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2									
> ♥ F_2_2[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2									
> ♥ F_2_3[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2									
> ♥ F_3_1[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2									
> ♥ F_3_2[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2									
> ♥ F_3_3[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2									
> ♥ M_1_1[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510							
> ♥ M_1_2[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510							
> ♥ M_1_3[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510							
> ♥ M_2_1[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510							
> ♥ M_2_2[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510							
> ♥ M_2_3[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510							
> ♥ M_3_1[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510							
> ♥ M_3_2[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510							
> ♥ M_3_3[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510							